

Assignment 1: Solving 3-SAT with Tree Search

Introduction to AI, Instructor: Liaw Rung Tzuo | 廖容佐

Antony Wilson | 魏彦晨 (412262319)

April 01, 2025

1 Problem Definition

This assignment addresses the **3-SAT problem**, a special case of the *circuit satisfiability problem*. Given a set of boolean variables $\{x_i\}_{i=1,\dots,D}$, the goal is to find an assignment $\mathbf{x}^*$ that satisfies a Boolean formula expressed in conjunctive normal form:

$$f(X) = \bigwedge_{j=1}^m (s_{j,1}x_{\pi[j,1]} \vee s_{j,2}x_{\pi[j,2]} \vee s_{j,3}x_{\pi[j,3]}) \quad (1)$$

where $s_{j,k}$ indicates the literal's sign, and $\pi[j,k]$ denotes the index of the k^{th} literal in the j^{th} clause. The task is to implement tree search algorithms (*depth-first search*, *best-first search*, and *A* search*) to find $\mathbf{x}^*$.

2 Conceptual Approach

When solving a 3-SAT problem, a vector of specific boolean values that satisfies the given boolean formula must be found. A search algorithm can be used in this case to iterate through different possible solutions, testing them against the boolean expression until a correct one is found. Fundamentally, this can be approached in two different ways:

Incomplete solver: The initial state is a random vector of size D that has assigned a boolean value to each variable. Then, to generate other states, variables are toggled. In this case, nodes would be structured in a graph, which connects each node to D other nodes, each differing in one place from the original node. A search algorithm could then traverse this graph and find a solution. While giving the algorithm the option to choose which variable to change next could have advantages in a heuristic solver, the solver is incomplete and neither guarantees finding a solution nor can it prove unsatisfiability well.

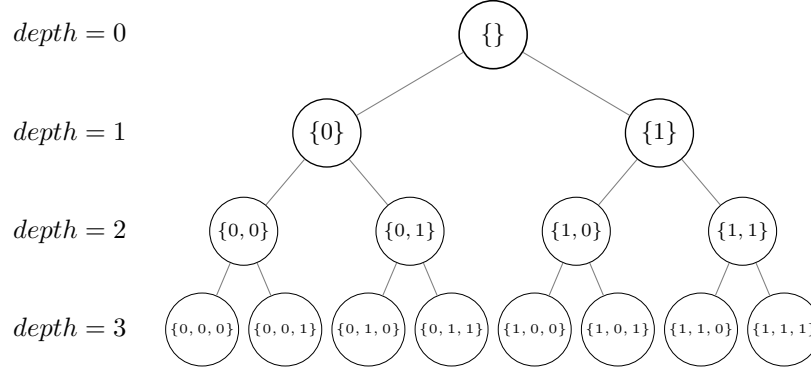


Figure 1: Search tree with $D = 3$. Each node represents a state of $\mathbf{x}$.

Complete solver: Another approach is to search in the form of a binary tree, shown in Figure 1. One starts with an empty solution set and assigns variables one by one until a vector of size D is reached, which is ideal for backtracking and helpful since the tree has a maximum depth of D , and no node is ever repeated. Search algorithms can easily be implemented without having to worry about endless looping. If a solution exists, it will be found if given enough time. Otherwise, unsatisfiability is proven, as all possible states are systematically exhausted.

For these reasons, I implemented a complete solver for all three algorithms.

Given the binary tree structure, it is clear that the **total number of states** is:

$$\text{All states} = 2^{D+1} - 1$$

and the **number of final states** (states at depth D) is:

$$\text{Final states} = 2^D$$

As D gets larger, the number of states grows exponentially (see Figure 2), making it important to implement a good search algorithm.

3 Implementation

The assignment asks for a way to implement the three different search algorithms that find a solution to the problem and then compare their performance. At this point, one should mention that this kind of satisfiability problem does not naturally lend itself to being solved by search algorithms such as GBFS and A* search. These are primarily graph traversal and pathfinding algorithms for finding optimal paths with minimum cost. While we can later use some characteristics of the problem to create usable heuristics for these algorithms,

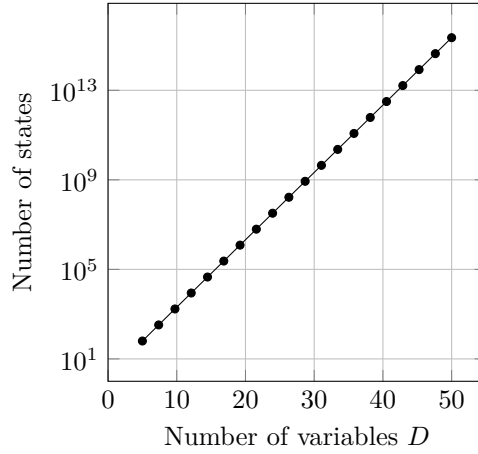


Figure 2: Total number of possible states of $\mathbf{x}$ as number of variables increases.

there is technically no “minimum cost path” to be found. Any solution that satisfies the problem is an acceptable result, as soon as it is found.

3.1 Depth-First Search (DFS)

Depth-first search is an uninformed search that immediately expands to the deepest node. In this case, we essentially use DFS to iterate through all possible permutations of the solution vector. No depth limit needs to be manually set, as the maximum depth of the graph is D . Since a solution cannot be at a depth less than D , the algorithm only needs to check a given state at depth D . Due to the simple structure of the binary tree, new states can be generated from the current state using recursive backtracking. This way, neither a concept of cost nor an explicit node structure needs to be implemented. At any given point, the program stores only one state in memory, and the call stack is at maximum D calls deep. The downside is, of course, that many states that could have been predicted at an earlier depth to evaluate as `false` still get expanded and checked. Especially when a given boolean formula turns out as unsatisfiable, all 2^D final states will be checked, which, as test results will show, quickly becomes incomputable as D becomes larger.

3.2 Greedy Best-First Search (GBFS)

The simple DFS iteratively expands nodes without checking whether a given node appears to be close to the goal or not. Greedy best-first search can be implemented, such that at any given point, the algorithm expands the node that “seems” closest to the goal based on a cost:

$$f_{\text{GBFS}}(\mathbf{x}) = h(\mathbf{x}) \quad (2)$$

If we think about how the 3-SAT problem works, it becomes apparent that the clauses of the boolean expression can be checked individually. Even though in the end all clauses must evaluate to **true**, it is clear that some clauses will do so before all variables are assigned. Furthermore, if any of the clauses evaluate to **false**, the whole formula is guaranteed never to evaluate to **true** no matter what variables are additionally assigned. As mentioned earlier, this problem does not have an obvious “path” that can be optimized. However, the number of clauses that are not yet satisfied can be seen as a path a heuristic function tries to minimize. While it is true that the goal will always be at depth D , the algorithm can use the heuristic to choose a node that is “closer” to the goal in the sense that it satisfies more clauses. As described in Equation (1), the formula has m clauses that need satisfying. Therefore, the number of unsatisfied clauses equals m minus the number of satisfied clauses. Using definitions from Equation (1), we can formulate the heuristic as:

$$h(\mathbf{x}) = m - \sum_{j=1}^m \text{eval}(s_{j,1}x_{\pi[j,1]} \vee s_{j,2}x_{\pi[j,2]} \vee s_{j,3}x_{\pi[j,3]}) \quad (3)$$

The function $\text{eval}(\cdot)$ returns an integer value based on the truth value and completeness of a given clause c :

$$\text{eval}(c) = \begin{cases} 1, & \text{if the clause is satisfied (at least one literal is **true**)} \\ 0, & \text{if the clause is incomplete (undef. literals } \geq 1, \text{ none **true**)} \\ -\infty, & \text{if the clause is unsatisfied (all literals are **false**)} \end{cases} \quad (4)$$

Since each satisfied clause subtracts 1 from the cost, the root node starts with a cost of m and a fully satisfying solution has a cost of 0. Note that if any clause is unsatisfied, the cost jumps to infinity. The given node will then automatically be sorted to the end of the priority queue, or an extra check can discard it immediately, such that it does not occupy space in memory. I implemented the latter.

3.3 A* Search

Usually, in a pathfinding problem, A* search can be directly implemented such that the cost of a given node:

$$f_{A^*}(\mathbf{x}) = g(\mathbf{x}) + h(\mathbf{x})$$

is an estimate of the cost of the whole path to the goal by taking the sum of the remaining cost predicted by the heuristic $h(\mathbf{x})$ and the cost $g(\mathbf{x})$ that it took to reach the current node. Among the three algorithms included in this assignment, interpreting the 3-SAT problem in a way that allows solving with A* search is the most tricky. The nature of A* search is to find an “optimal” or “cheapest” path to the goal node. However, the node structure in this problem is, by definition, such that every possible solution node has the same depth or “cost”

Input	Implementation	Time (ms)	Memory ¹	Expanded Nodes	Visited Nodes
$D = 10$	DFS	16	1	1 178	1 178
	GBFS	2	7	32	62
	A* ($\alpha = 0.87$)	2	9	18	34
$D = 20$	DFS	7 214	1	659 632	659 632
	GBFS	4	13	53	104
	A* ($\alpha = 0$)	4	13	53	104
$D = 30$	DFS	–	–	–	–
	GBFS	2 152	21	32 233	64 464
	A* ($\alpha = 0.2$)	1 907	21	29 136	58 270
$D = 40$	DFS	–	–	–	–
	GBFS	5 897	22	73 474	146 946
	A* ($\alpha = 0.87$)	1 399	1 526	15 766	31 530
$D = 50$	DFS	–	–	–	–
	GBFS	–	–	–	–
	A* ($\alpha = 0.87$)	57 292	353 553	542 462	1 084 922

Table 1: Measured performance of DFS, GBFS, and A* across different input sizes.

D . Nonetheless, if certain nodes satisfy clauses at a shallower depth than other nodes, this could indeed indicate a higher chance of leading to fully satisfying solutions. If clauses are satisfied early on, then there are more unassigned variables that can, when assigned, satisfy the remaining clauses. One can use the same definition in Equation (3) for $h(\mathbf{x})$. The cost of the current path $g(\mathbf{x})$ is proportional to the number of variables d that have been assigned. To match the current path cost $g(\mathbf{x})$ with the previously defined heuristic $h(\mathbf{x})$, we can normalize d , originally in the range $[0, D]$, to the range $[0, m]$ by scaling it proportionally:

$$g(\mathbf{x}) = \alpha \cdot d \cdot \frac{m}{D} \quad (5)$$

Due to the somewhat arbitrary nature of the function, I introduced a weight α that can be tweaked for optimal results. If, for example, $\alpha = 1.0$, then $g(\mathbf{x})$ approaches m as the program gets closer to the goal. This might cause the algorithm to avoid expanding a node at a depth close to D , even if $h(\mathbf{x})$ is very small.

4 Performance Comparison

For this assignment, only five data files for testing the program were provided. As described in Section 1, the files contain a matrix that represents the boolean

¹Maximum number of nodes stored in memory at a given point at runtime.

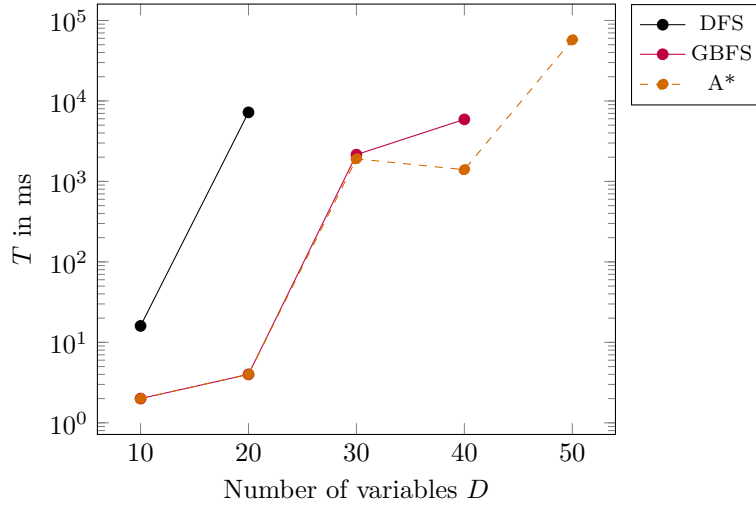


Figure 3: Runtime of DFS, GBFS and A* search.

formula. Each of the files increases the number of variables by 10, starting from 10 up to 50. I tested each of the three algorithms against each input file and measured the runtime, memory, total expanded nodes and total visited nodes. My measurements can be seen in Table 1, with blank fields meaning that the execution time was too long for me to measure (> 10 minutes).

Since DFS iterates through and expands all possible states, it quickly suffers from the exponential nature of the problem. Already at $D = 20$, I measured more than 659 632 expanded nodes and a runtime of more than 7 seconds. Note that since DFS expands every node it visits, the number of visited and expanded nodes is the same. At $D = 30$, DFS was unable to give any result within 10 minutes. Additionally, based on the jump in runtime from $D = 20$ to $D = 30$ with the other two algorithms, it is safe to say that the test data for $D = 20$ was in some way easier or quicker to solve. In Figure 3, I plotted the runtime of each algorithm, and it is apparent that, even at $D = 10$ and $D = 20$, DFS is vastly more inefficient than the other two algorithms. The one advantage of DFS is that it only ever stores one state and therefore uses the least amount of memory.

Comparing GBFS and A* search is somewhat more difficult. While it is clear that they both have an increased performance over DFS, it is harder to tell, especially at a lower number of variables, if one is better than the other. In my tests, GBFS solved all problems up to and including $D = 40$. At this point, it was expanding almost a hundred thousand nodes before it found the solution. During my first round of testing, A* search showed equal performance. Only after spending a lot of time tweaking α was I able to get better results. The performance observed when using A* search was, unfortunately, quite unpredictable. By incrementally changing α and repeated testing, I was able to

find values for α that resulted in better performance than GBFS. However, the values were not the same for each input. For $D = 10$, $D = 40$, and $D = 50$, $\alpha = 0.87$ made the program expand the minimum amount of nodes before finding the solution. However, for $D = 30$, A* and GBFS showed mostly the same performance, with A* being only negligibly better at $\alpha = 0.2$. For $D = 20$, I found no value of α that would improve A* search's performance over GBFS. Test results from $D = 40$ and $D = 50$ show that the non-greedy way with which A* decides which node to expand next results in more memory being used than GBFS, as more nodes are stored and considered in the priority queue.

5 Conclusion

It is clear that even though my heuristic function likely requires improvement with more testing and fine-tuning, A* search was able to find a solution for $D = 50$ within a reasonable time, whereas GBFS could not, and DFS already failed at $D = 30$. For solving the 3-SAT problem, A* search is the superior algorithm among the three, and a better-tuned heuristic function would likely show even better performance than my tests revealed. In my approach to this assignment, I did not try implementing an incomplete solver using A* search. It is possible that, with a well-defined heuristic function, an incomplete solver could show better performance.